浙江大学

ZHEJIANG UNIVERSITY

人工智能
实验报告

实验名称 FinalWork：CaptchaAgent

姓 名 、 、 、

实验日期 June 14, 2026

指导老师 杨倩倩

Contents

1 摘要 **1**

2 项目背景和问题定义 **1**

2.1 项目背景 1

2.2 问题定义 2

3 系统总体设计 **2**

3.1 总体架构 2

3.2 模块划分 3

3.3 系统技术栈 4

4 模块一：验证码类型分类 **4**

4.1 网络架构与流程 4

4.2 理论依据 5

4.3 训练与评价指标 6

4.4 误差来源 7

5 模块二：数字/字母验证码识别 **7**

5.1 网络架构与流程 7

5.2 理论依据 8

5.3 训练方式 9

5.4 评价指标和结果分析 11

5.5 误差来源 11

6 模块三：文字点选验证码识别 **12**

6.1 网络架构与流程 12

6.2 YOLOv8 文字区域检测 13

6.3 提示字与候选字分流 14

6.4 ArcFace 特征匹配 15

6.5 训练评价指标和结果 17

6.6 误差来源 19

7 结果的可视化分析 **19**

8 局限性 **21**

9 总结 **21**

A 附录：项目整体代码架构 **22**

B 附录：核心推理伪代码 **25**

1 摘要

本项目实现了一个面向多类型验证码的智能识别系统 CaptchaAgent。系统支持三类核心任务：验证码类型自动判断、数字/字母验证码识别，以及文字点选验证码识别。与单一模型直接输出结果的方式不同，本项目将验证码识别拆分为感知、路由、专用模型推理、结果解释和可视化展示等多个阶段，从而形成一个可运行、可分析、可扩展的人工智能应用系统。

系统总体上由三类 AI 模块组成。第一类是验证码类型分类模块，使用 ResNet18 对输入图像进行二分类，判断其应进入普通文字验证码识别流程还是文字点选验证码识别流程。第二类是数字/字母验证码识别模块，采用 CRNN 网络，将 CNN 特征提取、BiLSTM 序列建模和 CTC 解码结合起来，完成无字符级对齐标注条件下的字符串识别。第三类是文字点选验证码识别模块，先使用 YOLOv8n 检测文字区域，再通过提示/候选分流策略区分提示字和候选字，最后利用 ArcFace 特征嵌入、余弦相似度矩阵和贪心匹配恢复点击顺序。

从人工智能知识角度看，项目覆盖了监督学习、卷积神经网络、残差网络、序列建模、目标检测、度量学习、CTC 序列解码、置信度决策、启发式搜索和可解释性分析等内容。工程上，系统使用 FastAPI 封装后端推理服务，使用 React 前端展示识别结果、点击坐标、可视化图和推理日志。报告将按照系统总体设计、三个核心模块、训练与评价、误差来源、可视化分析、局限性和总结的顺序展开。

关键词： 验证码识别；CaptchaAgent；CRNN；CTC；YOLOv8；ArcFace；度量学习；可解释人工智能

2 项目背景和问题定义

2.1 项目背景

验证码是一种常见的人机验证机制，其目标是提高自动化程序批量访问系统的成本。传统验证码多为数字或字母组成的静态图片，识别问题可以近似看作字符序列识别。随着验证码形式发展，点选类验证码也变得常见：系统给出若干提示字，用户需要在图片中按顺序点击对应文字。这类验证码不仅要求识别字符，还要求检测目标位置并恢复点击顺序。

本项目在课程实验范围内构建一个多类型验证码识别系统，并不面向真实网站的安全绕过，而是将验证码作为综合 AI 应用案例，研究如何把图像分类、序列识别、目标检测和度量学习组织成一个完整系统。系统输入为验证码图像，输出为识别字符串或点击坐标序列，同时给出推理过程、置信度和可视化分析结果。

2.2 问题定义

设输入验证码图像为 I。系统首先需要判断验证码类型：

$$c = f_\theta(I), \quad c \in \{\text{text}, \text{click}\}.$$

其中 f_θ 为类型分类器，输出类别及置信度。

当 $c = \text{text}$ 时，任务是预测一个字符序列：

$$y = (y_1, y_2, \cdots, y_T), \quad y_i \in \mathcal{A}.$$

字符集合 $\mathcal{A}$ 包含数字、大写字母和小写字母。该任务的关键难点在于字符位置不固定、字符可能粘连、图像中存在干扰线和噪声。

当 $c = \text{click}$ 时，任务是输出一组按提示顺序排列的点击坐标：

$$P = ((x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)).$$

该任务同时包含文字区域检测、提示字与候选字分流、字符相似度匹配和点击顺序恢复四个子问题。

3 系统总体设计

3.1 总体架构

CaptchaAgent 采用模块化设计。系统可以分为四层：前端交互层、后端服务层、Agent 调度层和模型推理层。前端负责验证码生成、图片上传、识别结果展示和历史记录管理；后端负责图像解码、接口封装和结果格式化；Agent 调度层负责类型判断、模型路由和推理日志记录；模型推理层包含类型分类器、CRNN 文字识别器和点选验证码识别器。

整体流程为：用户上传或生成验证码图像；后端接收图像并传入 CaptchaAgent；Agent 在自动模式下先调用类型分类器；若分类置信度足够高，则直接调用对应识别器；若置信度较低，则可同时尝试多个识别器并依据结果置信度进行选择。最后系统返回识别结果、置信度、点击坐标、可视化图和推理日志。



Figure 1: CaptchaAgent 前端首页界面

图 1 展示了系统面向用户的入口界面。界面将验证码生成、上传识别、结果展示和历史记录组织在同一工作流中，说明本项目不仅完成模型训练，还将模型封装为可交互的应用系统。前端界面的作用是降低模型调用门槛，并为后续推理日志和可视化结果提供展示载体。

3.2 模块划分

为了降低系统耦合度，本项目将三个 AI 功能模块分开设计、训练和评价。

Table 1: CaptchaAgent 核心模块划分

模块	输入/输出	主要方法
验证码类型分类	输入整张验证码图像，输出 text/click 类型及置信度	ResNet18 二分类、softmax 置信度、Agent 路由
数字/字母验证码识别	输入普通文字验证码，输出字符串	CNN 特征提取、BiLSTM 序列建模、CTC 解码
文字点选验证码识别	输入点选验证码，输出点击坐标序列	YOLOv8 检测、提示/候选分流、ArcFace embedding、余弦相似度、贪心匹配

这种划分使每个模块都有明确任务边界。分类器只负责路由，不参与字符识别；CRNN 只处理普通文字验证码，不负责点击坐标；点选模块只处理检测和匹配，不依赖 CRNN 的字符序列输出。模块之间通过规范化输入输出连接，从而降低耦合度。

3.3 系统技术栈

Table 2: 系统主要技术栈

部分	技术与作用
前端	React 18、Vite、CSS，用于验证码生成、上传、结果展示和历史记录
后端	FastAPI、Pydantic、uvicorn，用于接口封装和请求响应管理
图像处理	OpenCV、Pillow、NumPy，用于图像读写、裁剪、预处理和可视化
深度学习	PyTorch、torchvision，用于 ResNet18 分类器和 CRNN 模型
目标检测	Ultralytics YOLOv8，用于点选验证码文字区域检测
特征推理	ONNX Runtime，用于 ArcFace 模型的轻量化推理

4 模块一：验证码类型分类

4.1 网络架构与流程

类型分类模块的目标是判断输入验证码属于普通文字验证码还是文字点选验证码。模型采用 ResNet18 作为骨干网络，并将最后的分类层替换为二分类头：

$$\text{ResNet18} \rightarrow \text{Dropout} \rightarrow \text{Linear}(512, 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(256, 2).$$

模型输出两个 logits，经 softmax 后得到 text 与 click 两类概率。Agent 读取概率较大的类别作为预测类型，同时将该概率作为路由置信度。

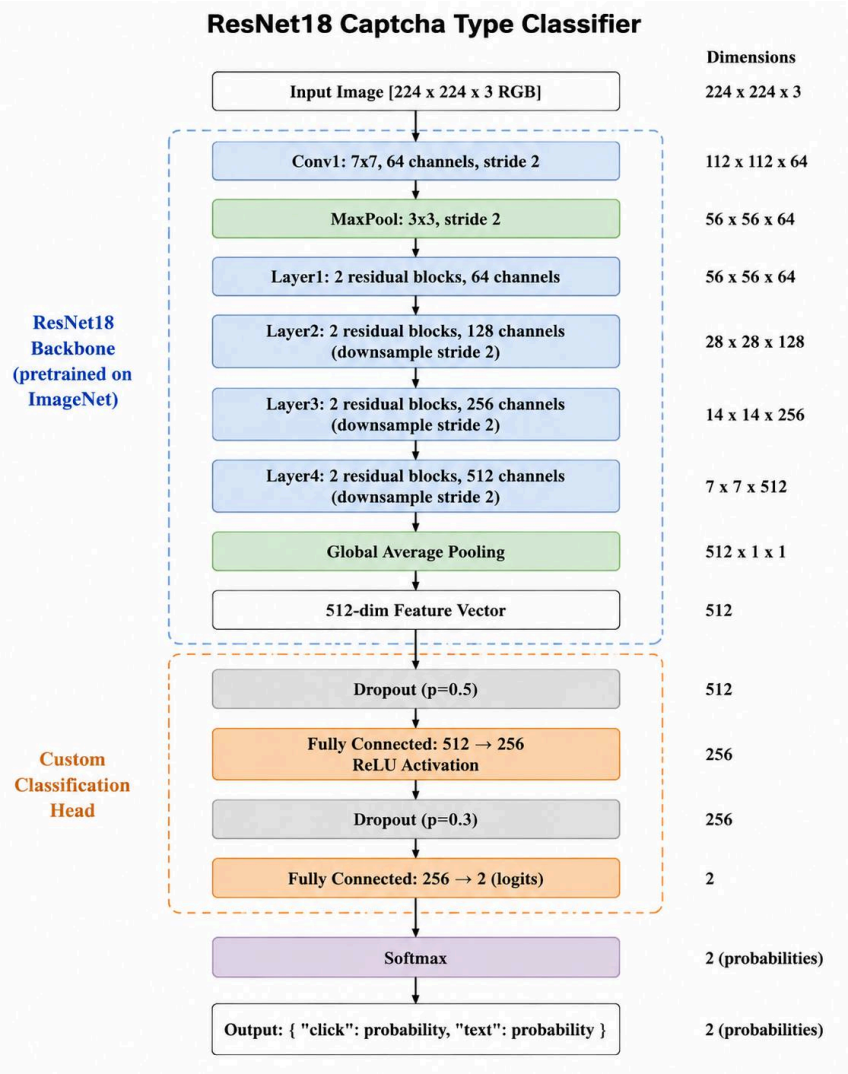


Figure 2: 验证码类型二分类器训练与推理流程

图 2 展示了类型分类器从图像预处理、ResNet18 特征提取到二分类输出的流程。该图强调分类器在系统中的定位：它不是最终识别模型，而是 Agent 调度前的路由模块。分类器输出的置信度会直接影响后续调用 CRNN 还是点选识别流水线。

4.2 理论依据

该模块对应监督学习中的图像分类问题。训练数据由验证码图像和类别标签组成，模型通过交叉熵损失函数学习类别判别边界。ResNet18 中的残差连接缓解了深层网络训练中的梯度退化问题，使模型能够学习更稳定的视觉特征。

从系统角度看，类型分类器承担的是“感知和路由”功能。它不是最终识别器，而是决定后续调用哪个专家模型。因此，该模块体现了多模型系统中的门控机制和置信度驱动决策思想。

4.3 训练与评价指标

类型分类器训练时将 text 和 click 两类验证码作为监督标签，输入图像统一缩放并归一化后送入 ResNet18。训练任务是标准二分类任务，因此损失函数选择交叉熵损失：

$$\mathcal{L}_{cls} = -\sum_{k=1}^{2} y_k \log p_k.$$

交叉熵适合 softmax 分类任务，能够直接惩罚真实类别概率过低的情况，因此适合本模块的 text/click 二分类目标。

Table 3: 验证码类型分类器训练配置

参数	取值
训练轮数	10 epochs
Batch size	32
初始学习率	0.001
Weight decay	0.0001
Dropout	0.5
优化器	Adam
学习率调度	StepLR，step size 为 10，gamma 为 0.1
损失函数	CrossEntropyLoss
最优模型选择	保存验证集准确率最高的 checkpoint

训练过程中使用 `CrossEntropyLoss` 作为损失函数，并在每个 epoch 后计算验证集准确率，保存表现最好的模型。

当前导出的分类器权重文件大小约为 135.84 MB（约 129.55 MiB）。实验中记录的最佳验证准确率为 100.00%，对应第 1 个 epoch。由于该模型用于 Agent 路由，除准确率外，还应关注置信度校准情况和低置信度样本比例。

评价指标包括分类准确率、混淆矩阵、平均置信度和低置信度样本比例。由于分类结果会直接影响后续路由，实际系统中还应关注误路由率，即 text 样本被送入 click 识别器或 click 样本被送入 text 识别器的比例。

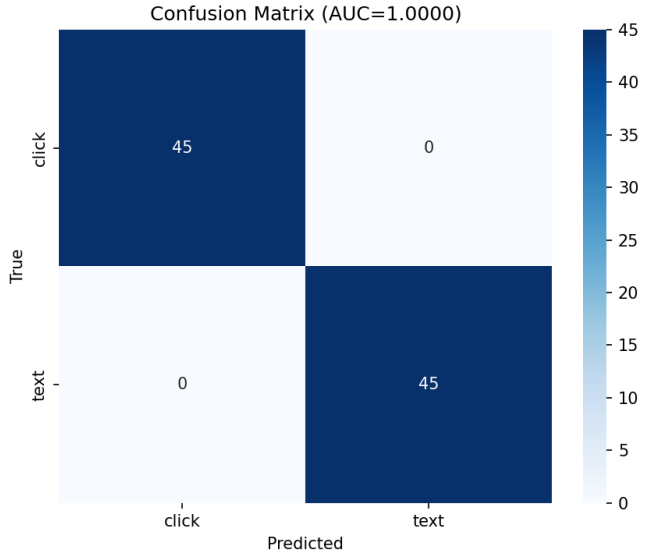


Figure 3: 验证码类型分类器混淆矩阵

图 3 用于观察 text 与 click 两类验证码之间的误判情况。若混淆矩阵主对角线占比高，说明分类器能够稳定区分两种验证码形态；若出现明显非对角元素，则表示 Agent 可能发生错误路由，使后续专家模型接收到不匹配的输入类型。

4.4 误差来源

分类错误主要来自两类情况。第一，验证码背景或布局相似，使 text 与 click 在整体视觉上难以区分。第二，输入图像经过缩放或裁剪后丢失了提示栏、候选字等关键结构。为降低误差传播，Agent 在低置信度场景下可以同时调用两个识别分支，并根据结果质量进行二次选择。

5 模块二：数字/字母验证码识别

5.1 网络架构与流程

数字/字母验证码识别模块采用 CRNN 结构。输入为灰度图像 $[B, 1, 60, 160]$。CNN 特征提取器由 5 个卷积阶段组成，通道数依次为 $1 \to 32 \to 64 \to 128 \to 256 \to 256$。前四个阶段使用 `MaxPool2d(2,2)` 压缩空间尺寸，第五个阶段使用 `AdaptiveAvgPool2d((1,None))` 将特征图高度压缩为 1。

随后模型执行 squeeze 和 permute 操作：

$$[B, 256, 1, 10] \to [B, 256, 10] \to [10, B, 256].$$

得到的时间序列输入 2 层双向 LSTM。LSTM hidden size 为 256，因此双向拼接后每个时间步输出 512 维特征。之后经过 Dropout 和 Linear 层输出 63 类字符概率，包括 62 个数字/字母字符和 1 个 CTC blank。最后使用 log softmax 和 CTC greedy decode 得到最终字符串。

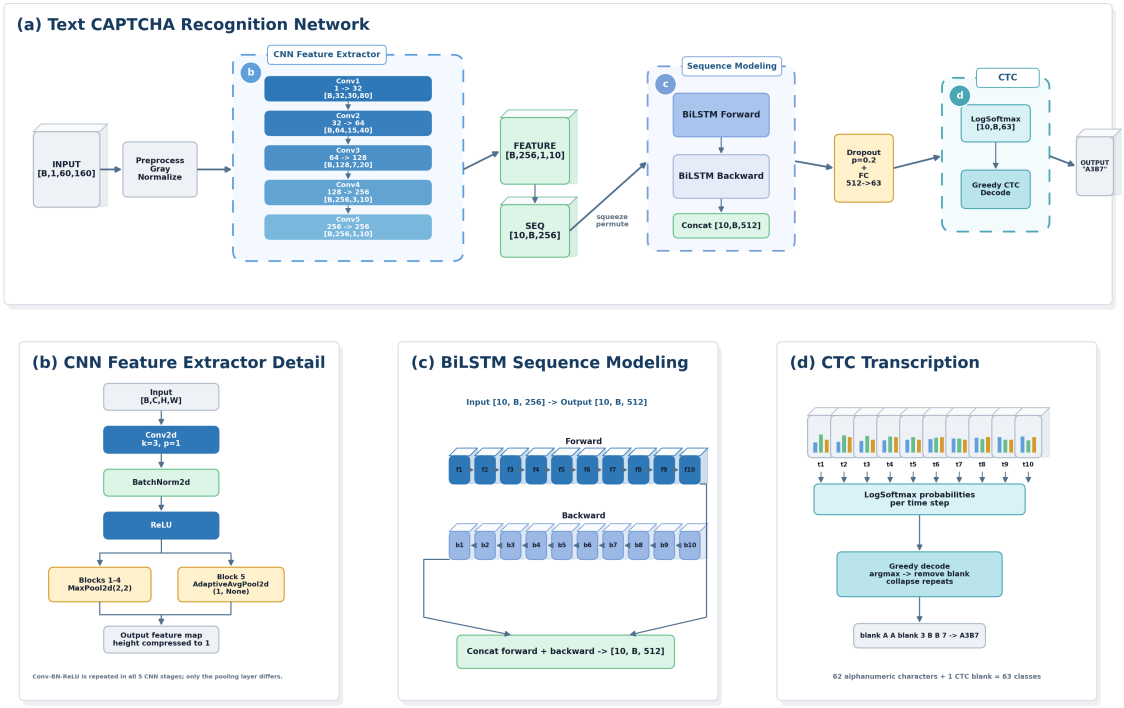


Figure 4: 数字/字母验证码 CRNN+CTC 识别流程

图 4 是数字/字母验证码识别模块的核心结构图。上方给出了从输入图像到 CTC 输出的主流程，下方分别放大 CNN 特征提取、BiLSTM 序列建模和 CTC 解码过程。该图能够直观看出模型如何把二维验证码图像压缩为宽度方向的时间序列，并通过 CTC 将逐时间步预测合并为最终字符串。

5.2 理论依据

该模块体现了序列识别中的三个关键思想。第一，CNN 将二维图像转换为更紧凑的视觉特征序列；第二，BiLSTM 同时建模从左到右和从右到左的上下文信息，有助于处理字符粘连和局部遮挡；第三，CTC 允许模型在没有字符级对齐标注的情况下训练序列识别器。

CTC 的 blank 类别用于表示“当前时间步不输出字符”。推理时，模型先取每个时间步概率最大的类别，再删除 blank 并合并连续重复字符。例如：

$$, A, A, , 3, B, B, 7 \rightarrow A3B7.$$

这使模型不需要知道每个字符在图像中的精确位置。

5.3 训练方式

文字验证码训练数据由程序自动合成。每张图像包含 4 位数字/字母字符，并加入字体变化、位置扰动、颜色变化、干扰线、噪声和轻微形变。合成数据的优点是标签天然准确，能够低成本生成大量训练样本。

训练时使用 CTC Loss。与普通分类不同，CTC Loss 不要求给出每个时间步对应的字符，只要求给出完整目标字符串。该任务中字符在图像中的具体位置不固定，如果使用逐位置交叉熵，就必须人工标注字符对齐位置，成本很高；CTC Loss 能自动枚举可能的时间步对齐路径，因此更适合验证码序列识别：

$$\mathcal{L}_{ctc} = -\log p(y|x).$$

模型输出为 $[T, B, C]$ 的 log probability，其中 $T = 10$，$C = 63$。本模块训练配置如表 4 所示。

Table 4: 数字/字母验证码 CRNN 训练配置

参数	取值
训练集 / 验证集 / 测试集	10000 / 2000 / 1000
Batch size	64
最大训练轮数	50 epochs
初始学习率	1×10^{-3}
Weight decay	1×10^{-5}
优化器	AdamW
学习率调度	Cosine scheduler，最小学习率 1×10^{-6}
早停策略	patience $= 3$，min delta $= 1 \times 10^{-4}$
梯度裁剪	max grad norm $= 5.0$
正则化	BiLSTM dropout $= 0.1$，输出 Dropout$(p = 0.2)$
损失函数	CTCLoss，blank index $= 0$

Table 5: 数字/字母验证码数据增强配置

增强项	取值
噪声点数量	20–80
干扰线数量	1–2
字体大小	22–48
旋转范围	$-5°$ 到 $5°$
字符间距扰动	8
垂直方向扰动	5

当前保存的最佳模型文件大小约为 43.75 MB（约 41.72 MiB），模型参数量为 3,628,005。训练过程共进行 20 个 epoch，best val loss 为 0.0370，best val accuracy 为 95.00%，最终学习率为 6.84×10^{-4}。从实验结果看，CRNN+CTC 已能够较稳定地完成 4 位数字/字母验证码的整串识别。

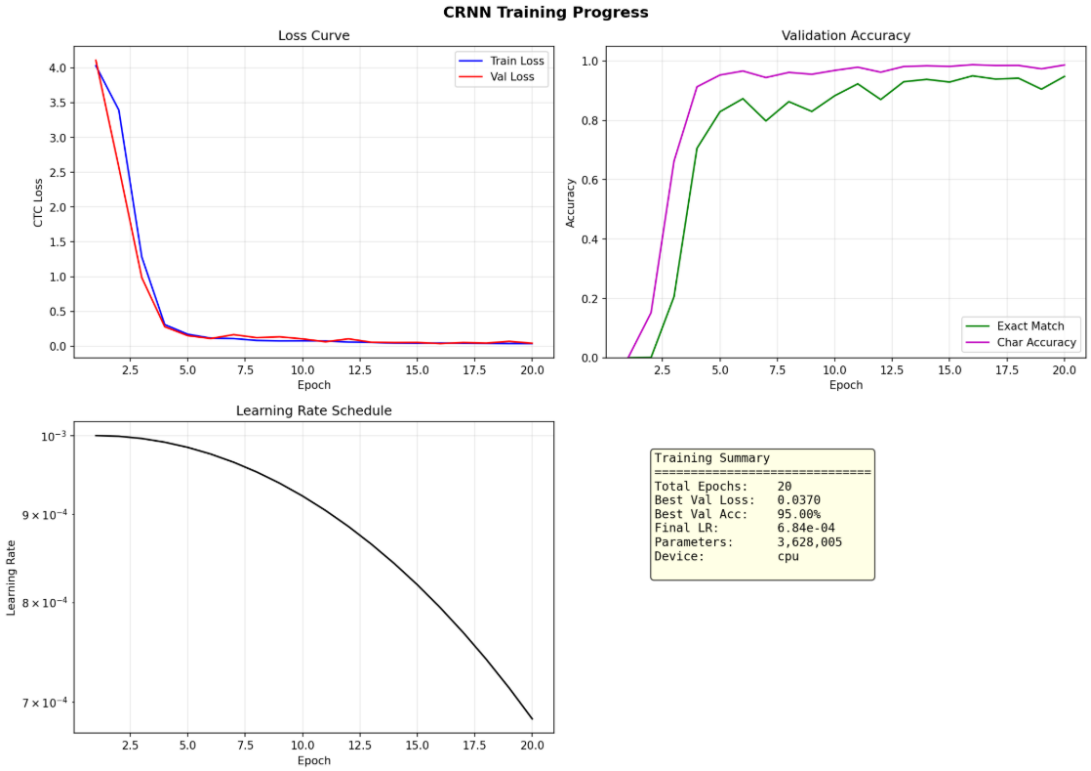


Figure 5: CRNN 文字验证码识别模型训练过程

图 5 展示了 CRNN 的训练损失、验证准确率和学习率变化。前 4 个 epoch 中训练集和验证集 CTC loss 快速下降，说明模型较快学习到图像特征与字符序列之间的对应关系；随后 loss 保持在较低水平并缓慢波动。验证集完整匹配率最终达到约 95%，字符级准确率接近 98% 以上。字符级准确率高于完整匹配率，是因为 4 位验证码中任意

一位预测错误都会导致整串匹配失败。学习率曲线体现了余弦退火调度在 20 轮训练中的平滑下降过程。

5.4 评价指标和结果分析

该模块应使用字符级准确率和整体验证码准确率两个指标。字符级准确率衡量单个字符预测是否正确；整体验证码准确率要求完整字符串完全一致。由于验证码任务只要一个字符错误就会导致最终失败，因此整体验证码准确率更能反映实际效果。

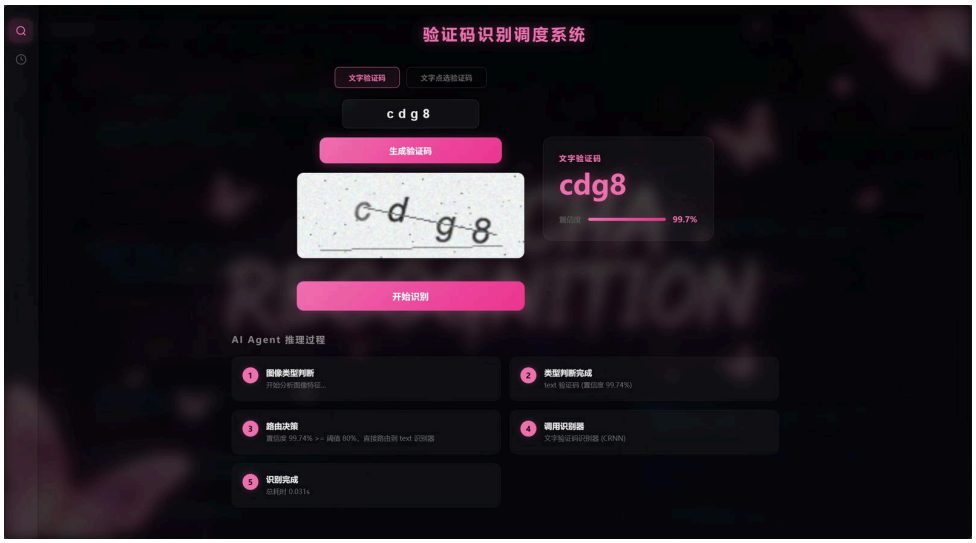


Figure 6: 数字/字母验证码识别结果示例

图 6 展示了数字/字母验证码的端到端识别效果。该图主要用于验证模型输出是否能够直接服务于前端应用：用户输入或生成验证码后，系统返回预测字符串和置信度。若预测结果与图像中字符一致，说明 CRNN+CTC 模块完成了从图像到文本的完整转换。

5.5 误差来源

该模块的错误主要来自字符粘连、强干扰线、字符旋转过大、相似字符混淆和 CTC 时间步不足。例如，字符 O 与 0、l 与 1、S 与 5 在低分辨率或强噪声下容易混淆。若 CNN 输出序列长度过短，也可能无法充分表示所有字符位置，从而影响 CTC 解码。

6 模块三：文字点选验证码识别

6.1 网络架构与流程

文字点选验证码识别模块的目标是输出提示字对应的点击坐标。该模块不是一个端到端单模型，而是一条由检测、分流、特征匹配和坐标恢复组成的流水线。

流程如下：首先使用 YOLOv8n 检测图像中的所有文字区域，得到边界框集合；然后使用边缘贴合、几何拓扑和 UI 容器三种策略区分提示字与候选字；接着分别裁剪提示字和候选字，并输入 ArcFace 特征提取器得到 256 维 L2 归一化向量；最后计算提示字和候选字之间的余弦相似度矩阵，通过贪心全局匹配恢复点击顺序并输出候选框中心坐标。

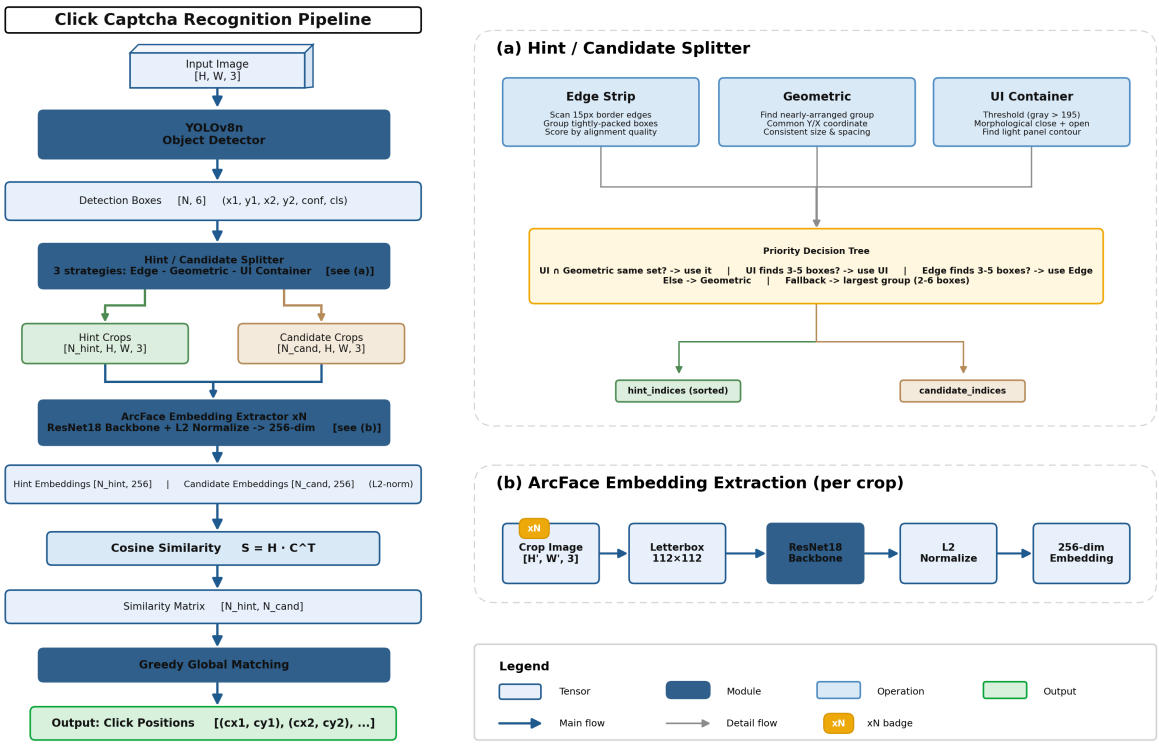


Figure 7: 文字点选验证码识别流程

图 7 展示了文字点选验证码识别的完整流水线。左侧为主流程，从输入图像、YOLO 检测、提示/候选分流、ArcFace embedding 到最终点击坐标；右侧放大了提示候选分流和单字 embedding 提取两个关键步骤。该图体现出点选验证码不是单一分类任务，而是检测、规则推理、度量学习和匹配算法组合而成的多阶段问题。

6.2 YOLOv8 文字区域检测

YOLOv8n 是单阶段目标检测模型，能够直接预测边界框、类别和置信度。本项目采用单类别检测策略，即只检测“文字区域”，不让 YOLO 判断某个框是提示字还是候选字。这样可以降低标注难度，并将版式相关的提示/候选分流交给后续规则模块处理。

YOLO 训练使用合成点选验证码数据。检测模型的训练配置如表 6 所示。

Table 6: YOLOv8n 文字检测训练配置

参数	取值
预训练权重	yolov8n.pt
任务类型	detect
输入尺寸	640×640
训练轮数	200 epochs
Batch size	16
Patience	50
初始学习率 lr0	0.001
最终学习率比例 lrf	0.01
Momentum	0.937
Weight decay	0.0005
Warmup epochs	5
AMP 混合精度	启用
保存周期	每 20 个 epoch 保存一次

Table 7: YOLOv8n 数据增强配置

增强项	取值
HSV 色调扰动	0.015
HSV 饱和度扰动	0.7
HSV 亮度扰动	0.4
旋转角度	5.0
平移比例	0.1
缩放比例	0.5
Mosaic	0.5
MixUp	0.2
Close mosaic	最后 10 个 epoch

当前导出的 YOLO 权重文件大小约为 6.26 MB（约 5.97 MiB）。

YOLOv8 的训练损失由边界框回归损失、分类损失和分布焦点损失组成。边界框损失用于优化预测框与真实框的位置重合程度，分类损失用于判断目标类别，DFL 用于提升边界框定位精度。虽然本项目只检测单一类别“文字区域”，分类损失仍有助于区分文字目标和背景区域。



Figure 8: YOLOv8 在验证集上的文字区域检测预测示例

图 8 展示了 YOLOv8n 在验证集上的检测结果。可以通过该图检查模型是否能覆盖提示字和候选字区域，以及是否存在漏检、重叠框或背景误检。对于点选验证码任务而言，检测框质量非常关键，因为后续提示/候选分流和 ArcFace 匹配都依赖这些裁剪区域。

6.3 提示字与候选字分流

检测出所有文字框后，系统需要判断哪些属于提示栏，哪些属于候选点击区域。本项目使用三种互补策略。

边缘贴合策略利用提示栏常位于边缘的先验，扫描图像边缘附近文字框，并根据对齐质量评分。几何拓扑策略利用提示字通常排列整齐的特点，根据坐标方差、间距一致性和尺寸一致性选择最可能的一组提示字。UI 容器策略利用提示栏可能具有浅色背景框的特点，通过阈值、形态学操作和轮廓筛选找到提示栏区域。

这一部分体现了神经网络与规则推理的结合：YOLO 负责提供候选目标，规则模块负责引入版式先验，从而提升系统可解释性和稳定性。

6.4 ArcFace 特征匹配

ArcFace 模块用于判断提示字和候选字是否为同一个汉字。每个裁剪图像经过 letterbox 到 112×112，再输入 ResNet18 backbone，最后输出 256 维 embedding。向量经过 L2 归一化后，可用点积计算余弦相似度：

$$s(h, c) = e_h^\top e_c.$$

所有提示字和候选字两两比较后形成相似度矩阵 S。系统每次选择当前最大相似度完成一组配对，并屏蔽对应行列，直到完成所有提示字匹配。

ArcFace 训练数据包含 266 个常用汉字类别。训练配置如表 8 所示。

Table 8: ArcFace 字符特征模型训练配置

参数	取值
字符类别数	266
每类样本数	500
Backbone	ResNet18
Embedding 维度	256
最大训练轮数	80 epochs
Batch size	32
初始学习率	0.01
Weight decay	5×10^{-4}
优化器	SGD，momentum = 0.9
学习率调度	CosineAnnealingLR
ArcFace scale	$s = 30.0$
ArcFace margin	$m = 0.35$
Margin warmup	5 epochs
损失函数	CrossEntropyLoss over ArcFace logits

ArcFace 的损失函数本质上是带角度间隔的 softmax 分类损失。普通 softmax 只要求样本被分到正确类别，而 ArcFace 在角度空间中加入 additive angular margin，使同

类样本特征更加聚集、异类样本间隔更大，因此更适合后续使用余弦相似度进行匹配。实验中使用 `CrossEntropyLoss` 计算 ArcFace head 输出 logits 与字符类别标签之间的损失。

实验中 ArcFace-ResNet18 的最佳模型出现在第 27 个 epoch，验证准确率达到 99.71%，类别数为 266，embedding 维度为 256。训练阶段保存的 PyTorch checkpoint 大小约为 91.05 MB（约 86.83 MiB）。实际推理时使用 ONNX 格式导出的 backbone，文件大小约为 45.19 MB（约 43.10 MiB）。

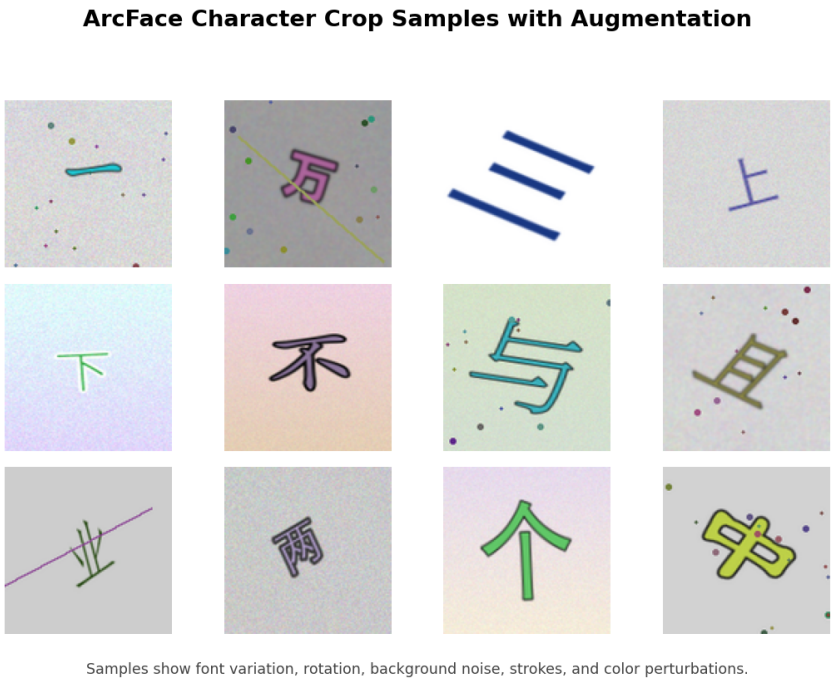


Figure 9: ArcFace 单字裁剪图训练样本及数据增强效果

图 9 展示了 ArcFace 训练时使用的单字裁剪图样本。该图的目的不是展示识别结果，而是说明训练数据中加入了字体变化、旋转、背景噪声、干扰线、颜色扰动和描边等增强方式。通过这些增强，模型学习到相对稳定的字符身份特征，而不是只记住某一种字体或背景。与直接 OCR 相比，ArcFace 更关注“两个裁剪图是否属于同一个字”，因此适合点选验证码中的提示字与候选字匹配。

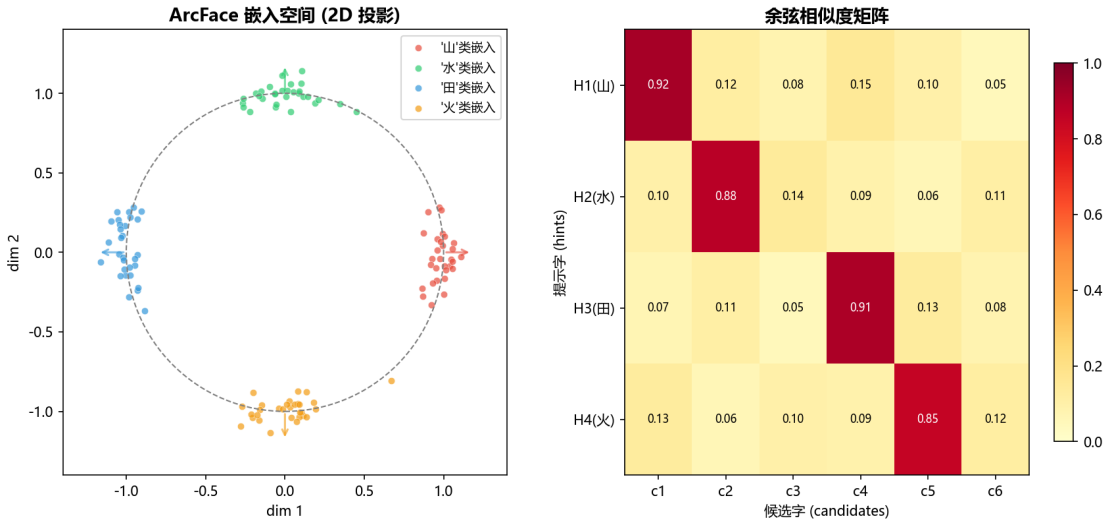


Figure 10: ArcFace 字符嵌入空间与相似度矩阵可视化

图 10 展示了 ArcFace 训练后特征空间和匹配结果的可视化效果。左侧二维投影中，同一字符类别的 embedding 聚集在相邻区域，不同类别之间保持明显间隔，说明角度间隔损失提升了类内紧凑性和类间可分性。右侧余弦相似度矩阵中，提示字与正确候选字之间的相似度明显高于其他候选字，这正是后续贪心匹配能够恢复点击顺序的依据。

6.5 训练评价指标和结果

点选验证码模块由 YOLO 检测器、提示/候选分流规则和 ArcFace 字符匹配模型共同组成，因此评价时需要分阶段观察。YOLO 部分关注文字框是否被准确检测，ArcFace 部分关注提示字与候选字是否能在特征空间中正确匹配，最终端到端结果则关注点击顺序和点击坐标是否正确。阶段性评价结果如表 9 所示。

Table 9: 文字点选验证码模块阶段性训练与评价结果

子模块	评价指标	实验结果
YOLOv8n 文字检测	precision / recall / mAP50 / mAP50-95	0.9996 / 1.0000 / 0.9950 / 0.9667
ArcFace 字符匹配	验证集分类准确率	99.71%
ArcFace 特征空间	类内紧凑性、类间可分性、余弦相似度间隔	同字相似度明显高于异字相似度，可支持提示字–候选字匹配
端到端点选流程	点击顺序、点击坐标、推理可解释性	通过检测框、匹配连线和推理日志进行综合验证

YOLO 检测部分使用 precision、recall、mAP50 和 mAP50-95。训练日志显示，YOLOv8n 在第 200 个 epoch 时 precision 为 0.9996，recall 为 1.0000，mAP50 为 0.9950，mAP50-95 为 0.9667，说明检测器能够较稳定地定位文字区域。

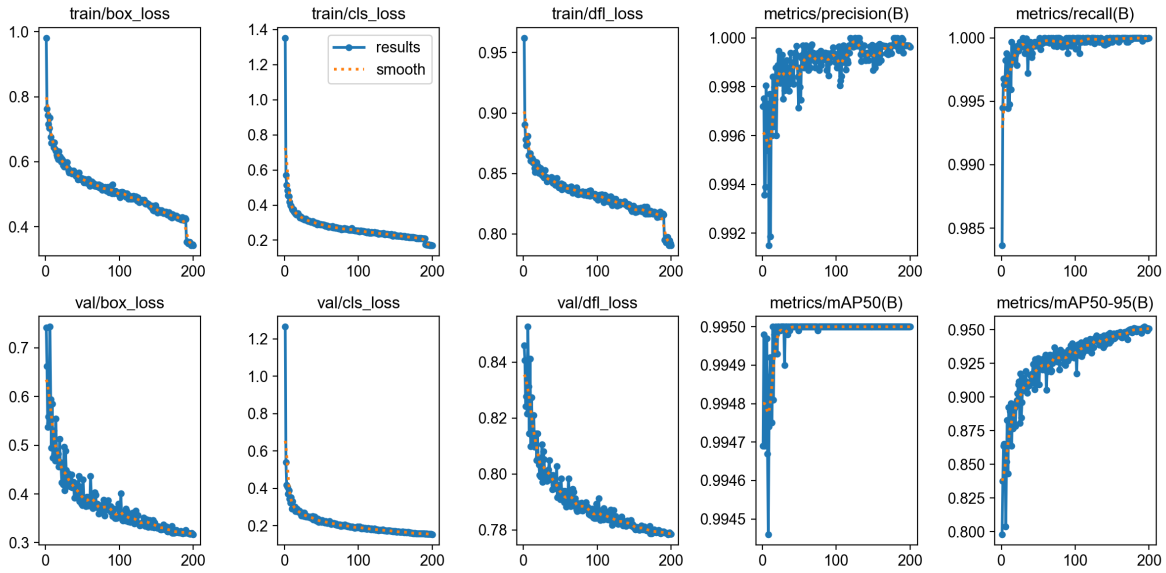


Figure 11: YOLOv8n 文字检测模型训练过程曲线

图 11 展示了 YOLOv8n 训练过程中的多项指标变化。训练集和验证集的 box loss、cls loss、DFL loss 均整体下降，说明模型的边界框定位、目标分类和框分布回归逐步收敛。precision 和 recall 在训练早期迅速接近 1，mAP50 基本保持高水平，而 mAP50-95 随 epoch 增长逐步提高，说明模型不仅能检测到文字目标，也在逐渐提升边界框定位精度。相比单类别检测中的混淆矩阵，这组训练曲线更能反映 YOLO 模型的训练质量。

ArcFace 部分使用分类准确率和 embedding 区分能力作为评价依据。实验中 ArcFace-ResNet18 在 266 个汉字类别上进行训练，最佳模型验证准确率达到 99.71%，说明模型能够较好地区分不同汉字类别。由于点选任务最终并不是直接输出类别编号，而是比较提示字和候选字的相似度，因此单纯分类准确率还不够，还需要观察 embedding 是否具有“同字更近、异字更远”的性质。图 10 中的二维投影和余弦相似度矩阵说明：同类字符特征在嵌入空间中聚集，不同字符之间保持间隔；正确匹配位置的余弦相似度显著高于错误匹配位置，因此 ArcFace 特征可以作为后续贪心匹配的可靠依据。

端到端点选模块还应使用点击顺序完全正确率、坐标平均误差和推理耗时作为最终评价指标。由于该流水线存在误差传递，若 YOLO 检测框准确但最终点击顺序错误，问题通常来自提示/候选分流或 ArcFace 相似度矩阵；若检测框本身缺失或偏移，则后续匹配即使正确也无法恢复正确点击坐标。因此本实验报告将检测曲线、ArcFace 特征可视化和最终点击结果图结合起来分析整体效果。

6.6 误差来源

点选验证码的误差可能来自四个阶段。第一，YOLO 漏检或误检文字框；第二，提示字与候选字分流错误；第三，ArcFace 对形近字或强形变字产生相似度混淆；第四，贪心匹配在局部相似度错误时导致全局顺序错误。由于该模块是流水线结构，早期错误会向后传播，因此调试时应分别检查检测框、分流结果、相似度矩阵和最终匹配路径。

7 结果的可视化分析

系统提供多种可视化结果。对于普通文字验证码，前端展示生成图像、识别字符串和置信度，使用户可以直接比较真实文本与预测文本。对于点选验证码，系统会绘制检测框、提示字和候选字匹配关系，并给出最终点击顺序。这些图像能够帮助判断错误来自检测、分流还是匹配阶段。



Figure 12: 文字点选验证码识别结果界面

图 12 展示了点选验证码识别结果在前端界面中的呈现方式。界面不仅显示最终点击坐标，还展示识别状态、置信度和可视化图像，方便用户检查模型是否按提示顺序完成匹配。该图说明点选识别模块已经被集成到完整应用流程中。

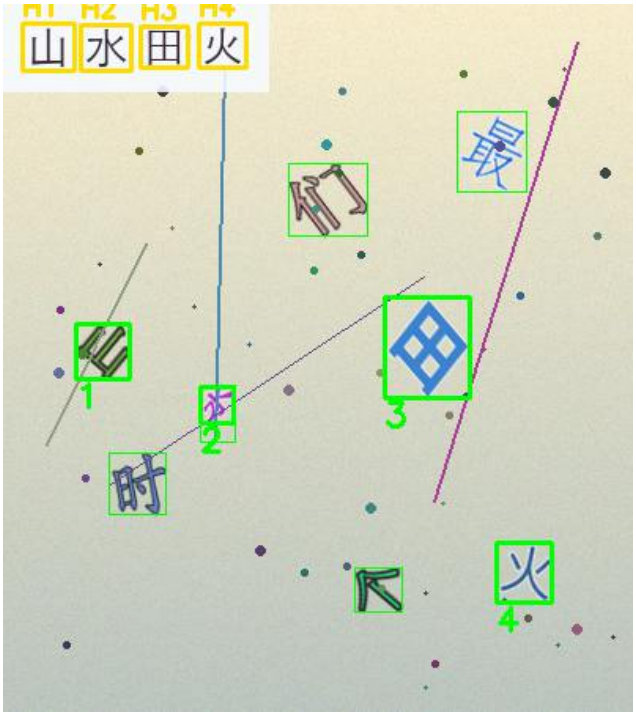


Figure 13: 文字点选验证码检测与点击顺序可视化

图 13 更具体地展示了检测框和点击顺序。通过观察候选框位置、编号顺序和最终连线，可以判断错误是否来自 YOLO 检测、提示/候选分流还是 ArcFace 匹配。如果检测框正确但顺序错误，则问题更可能出现在相似度矩阵或贪心匹配阶段。

此外，Agent 推理面板展示系统的中间决策过程，包括类型判断、模型路由、置信度、调用模型和耗时信息。这增强了系统的可解释性，使用户能够理解模型为什么给出当前结果。

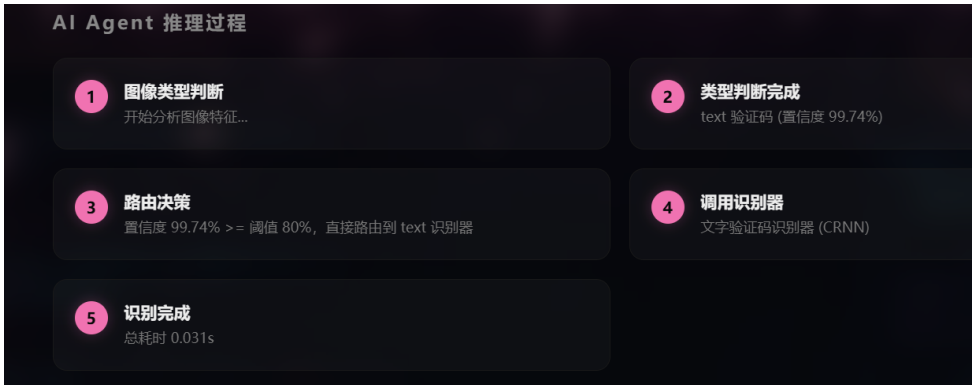


Figure 14: AI Agent 推理过程展示

图 14 展示了 Agent 的推理日志。该面板将验证码类型判断、模型路由、置信度和各阶段耗时记录下来，使系统不只是给出最终答案，还能解释“为什么调用该模块”和“识别过程经过哪些步骤”。这对于分析错误样本和改进模型具有实际价值。

8 局限性

当前系统仍有若干局限。首先，训练数据主要由程序合成，与真实网站验证码可能存在分布差异。其次，点选验证码模块依赖提示栏结构，如果提示栏位置、背景或排列方式变化较大，分流策略可能失效。第三，ArcFace 虽然在单字分类上表现较好，但对形近汉字、严重遮挡和极端字体仍可能产生混淆。第四，贪心匹配无法保证全局最优，当相似度矩阵中存在局部错误高分时，可能导致后续匹配连锁出错。第五，系统目前主要处理静态图像验证码，尚未覆盖滑块、多轮交互和语义选择类验证码。

9 总结

本项目实现了 CaptchaAgent 多类型验证码识别系统。系统通过 ResNet18 完成验证码类型判断，通过 CRNN+CTC 完成数字/字母验证码识别，通过 YOLOv8n、提示/候选分流、ArcFace embedding 和贪心匹配完成文字点选验证码识别，并通过 FastAPI 和 React 构建了端到端可交互应用。

相比只训练单个模型，本项目更强调模块化 AI 系统的设计。每个模块都有明确的输入、输出和评价指标，模块之间通过 Agent 进行路由和组合。报告中使用新的流程图展示了数字/字母验证码识别和文字点选验证码识别的具体流程，使系统结构更加清晰。整体来看，该项目较完整地覆盖了人工智能课程中的图像分类、目标检测、序列识别、度量学习、搜索匹配和可解释性等内容，具有较好的课程综合实践意义。

A 附录：项目整体代码架构

```
AI/
|-- README.md
|-- requirements.txt
|
|-- backend/                    # Python 后端
|   |-- main.py                 # FastAPI 入口 (端口 8765)
|   |
|   |-- app/                    # 核心应用代码
|   |   |-- generator.py        # 验证码图片生成器
|   |   |
|   |   |-- agent/              # Agent 调度核心
|   |   |   |-- scheduler.py    # CaptchaAgent 编排器
|   |   |   |-- classifier_cnn.py  # ResNet18 二分类器 (文字 vs 点选)
|   |   |   |-- classifier_rule.py # CV 启发式分类器 (无 GPU 可用)
|   |   |   |-- dataset.py      # 分类数据集 + DataLoader
|   |   |   `-- reasoning_logger.py # 推理步骤记录
|   |   |
|   |   |-- models/             # 识别器 (纯推理 + 网络架构)
|   |   |   |-- text_captcha/   # 文字验证码识别 (CRNN)
|   |   |   |   |-- recognizer.py  # 识别入口
|   |   |   |   |-- model.py       # CRNN 网络定义
|   |   |   |   |-- predict.py     # CaptchaPredictor 推理引擎
|   |   |   |   `-- config.py      # 推理常量 + 模型架构参数
|   |   |   |
|   |   |   `-- click_captcha/     # 点选验证码识别 (YOLO + ArcFace)
|   |   |       |-- recognizer.py  # 识别入口
|   |   |       `-- utils/
|   |   |           |-- arcface_onnx.py # ArcFace ONNX 推理封装
|   |   |           |-- matchingMode.py # 相似度矩阵 + 贪婪匹配
|   |   |           `-- hint_splitter.py # 提示字/候选字分流
|   |   |
|   |   `-- utils/              # 通用工具
|   |       `-- augment.py      # 数据增强 (train/val transform)
|   |
|   |-- models/                 # 模型权重 (集中管理, 不入 git)
|   |   |-- classifier/
```

```
|   |   |   `-- best_classifier.pth # ResNet18 二分类器
|   |   |-- text_captcha/
|   |   |   |-- best_model.pth     # CRNN 文字识别
|   |   |   `-- last_model.pth
|   |   `-- click_captcha/
|   |       |-- yolo/
|   |       |   `-- best.pt        # YOLO 文本检测
|   |       `-- arcface_backbone.onnx # ArcFace 特征提取 ONNX
|   |
|   |-- training/                  # 训练脚本 + 数据 (与推理分离)
|   |   |-- classifier/            # 二分类器训练
|   |   |   |-- train.py           # 训练入口
|   |   |   |-- split_dataset.py   # 数据集划分 (train/val/test)
|   |   |   |-- evaluate.py        # 模型评估 (混淆矩阵 + ROC)
|   |   |   |-- data/              # 分类器数据
|   |   |   `-- outputs/           # 评估产物
|   |   |
|   |   |-- text_captcha/          # CRNN 文字验证码训练
|   |   |   |-- train.py           # Trainer 类 + 训练入口
|   |   |   |-- train_config.py    # 训练超参数配置
|   |   |   |-- dataset.py         # 合成数据生成 + DataLoader
|   |   |   `-- train_script.py    # 备用训练脚本
|   |   |
|   |   |-- click_captcha/         # 点选验证码训练
|   |   |   |-- config.py          # 路径配置 + YOLO 训练超参
|   |   |   |-- yolo/
|   |   |   |   `-- train.py       # YOLO 检测训练
|   |   |   `-- data/
|   |   |       |-- synthesize.py  # 验证码图片合成器
|   |   |       |-- prepare_yolo.py # YOLO 标注格式转换
|   |   |       `-- backgrounds/   # 背景图片素材
|   |   |
|   |   `-- arcface_training/      # ArcFace 汉字分类预训练
|   |       |-- classify_train.py  # 训练入口 (含 ONNX 导出)
|   |       |-- arcface_model.py   # ResNet18 + ArcFaceHead
|   |       |-- classify_dataset.py # 分类数据集
|   |       |-- _export.py         # ONNX 导出工具
|   |       |-- config.py          # 路径配置
```

```
|   |       `-- data/
|   |           `-- generate_arcface_data.py
|   |
|   |-- tests/                    # 测试
|   |   |-- test_agent.py
|   |   |-- test_classifier.py
|   |   `-- test_click_captcha.py
|   |
|   `-- docs/                     # 文档
|       |-- TRAINING_GUIDE.md
|       |-- technical_report.md
|       `-- 文字验证码识别工具.md
|
`-- frontend/                     # React 前端 (Vite)
    |-- package.json
    |-- vite.config.js            # proxy /api -> localhost:8765
    |-- index.html
    |-- public/
    |   `-- bg.jpg                # 首页背景图
    `-- src/
        |-- main.jsx              # React 入口
        |-- App.jsx               # 主组件
        |-- App.css               # 全局样式
        |-- api/
        |   `-- mock.js           # API 客户端
        `-- components/
            |-- GeneratePanel.jsx # 验证码生成面板
            |-- UploadPanel.jsx   # 上传面板
            |-- ResultPanel.jsx   # 识别结果展示
            |-- ReasonPanel.jsx   # 推理过程展示
            `-- HistoryPanel.jsx  # 历史记录
```

B 附录：核心推理伪代码

Listing 1: CaptchaAgent 路由逻辑

```python
def route_and_recognize(image, mode="auto"):
    if mode == "auto":
        type_result = classifier.predict(image)
        captcha_type = type_result["type"]
        confidence = type_result["confidence"]
    else:
        captcha_type = mode
        confidence = 1.0

    if confidence >= 0.8:
        result = run_recognizer(captcha_type, image)
    else:
        text_result = run_recognizer("text", image)
        click_result = run_recognizer("click", image)
        result = choose_higher_confidence(text_result, click_result)

    return format_result(result)
```

References

[1] Pytorch documentation. https://pytorch.org/docs/stable/index.html. Accessed for framework reference.

[2] Ultralytics yolo documentation. https://docs.ultralytics.com/. Accessed for YOLOv8 usage reference.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[5] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.

[6] Baoguang Shi, Xiang Bai, and Cong Yao. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11):2298–2304, 2017.

[7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.

[8] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4690–4699, 2019.

[9] Onnx runtime documentation. https://onnxruntime.ai/docs/. Accessed for model inference deployment reference.